

COSC2396 (UNIX and Systems Programming in C)

Assignment 2 (Tic Tac Toe Game) Maximum

Bhavneet

239666750

Marks: 10

Game Strategy of Parent Process:

P0) Create a 3x3 (i.e. 9 boxes) board and initiate with empty spaces.

P1) Call a random number r between 0..8 to determine board(i, j) where $i=r/3$ and $j=r\%3$

P2) Find the current machine time (seconds elapsed since epoch January 1, 1970). Place X at board(i, j) if the current time in seconds is an even number and O if the current time in seconds is an odd number.

P3) Parent sends SIGUSR1 to child process requesting child's move.

P4) If it receives SIGUSR1 (or SIGUSR2) from child process, then it knows that child wants to place X (or O) as next move. Call a random number r between 0..8 to determine box for child's move board(i, j) where $i=r/3$ and $j=r\%3$. If the board(i, j) is already filled, find an empty box by inspecting the next boxes sequentially i.e. $r+1, r+2$, so on.

P5) Parent then needs to decide its own next move, essentially repeat from Step P1.

P6) Whenever there is a winner or there is a draw, parent sends SIGUSR2 to child process announcing the game over so that child process can gracefully exit.

Game Strategy of Child Process:

C1) Upon receiving SIGUSR1 from parent process, child tries to decide its next move.

C2) Find the current machine time (seconds elapsed since epoch January 1, 1970). Decides to place X if the current time in seconds is an even number and O if the current time in seconds is an odd number. Accordingly, sends SIGUSR1 for X and SIGUSR2 for O.

C3) Upon receiving SIGUSR2 from parent process, child tries to determine whether it was a win, loss or draw to plan for its celebration. Accordingly, it prints a message and terminate itself sending SIGTERM signal to itself. [Hint: parent makes 1st move, so it will make the 9th move]

Inside child's SIGUSR2 handler:

- Get current time (time (NULL) → seconds).
- If even → child chooses **X**, so it should restart by sending **SIGUSR1** to parent.
- If odd → child chooses **O**, so child sends **SIGUSR2** to parent.

Parent must interpret this:

- If parent receives SIGUSR1 → child wants replay; give child the first move using **X**.
- If parent receives SIGUSR2 → child wants replay with **O**.

Parent must **reset the board** to empty:

- 9 empty spaces again.
- Game loop starts fresh.

Pseudocode:→

```
while (1) {
    init_board();

    while(game_not_over) {
        parent_move();
        send_signal_to_child(SIGUSR1);

        pause(); // wait for child's signal
        take_child_move();
    }

    send_signal_to_child(SIGUSR2); // announce game over

    pause(); // Wait for child's replay decision

    if (received SIGUSR1) → child wants replay as X
    if (received SIGUSR2) → child wants replay as O
    // Reset game loop continues
}
```

Parent and child communicate using signals

- SIGUSR1 → normal move
- SIGUSR2 → game over / replay decision

✓ Parent controls the board

Child only picks position + symbol, parent stores it.

✓ Game restarts automatically after SIGUSR2

Depending on current second:

- Even → child sends SIGUSR1 (child wants X)
- Odd → child sends SIGUSR2 (child wants O)

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <time.h>

char board[9];
int childTurn = 0;    // 1 → child plays X, 2 → child plays O
pid_t child_pid;
int parent_signal = 0; // store child replay signal

void resetBoard() {
    for(int i=0;i<9;i++) board[i] = ' ';
}

void printBoard() {
    printf("\n");
    for(int i=0;i<9;i++) {
        printf(" %c ", board[i]);
        if((i+1)%3==0) printf("\n");
    }
}

int hasWinner() {
    int w[8][3] = {
        {0,1,2},{3,4,5},{6,7,8},
        {0,3,6},{1,4,7},{2,5,8},
        {0,4,8},{2,4,6}
    };
    for(int i=0;i<8;i++) {
        int a=w[i][0], b=w[i][1], c=w[i][2];
        if(board[a] != ' ' && board[a]==board[b] && board[b]==board[c])
            return 1;
    }
    return 0;
}

```

```

int boardFull() {
    for(int i=0;i<9;i++) if(board[i]==' ') return 0;
    return 1;
}

void parentMove() {
    int r = rand()%9;
    int t = time(NULL);
    char symbol = (t%2==0 ? 'X' : 'O');

    while(board[r] != ' ') r = (r+1)%9;
    board[r] = symbol;

    printf("Parent plays %c at %d\n", symbol, r);
    printBoard();
}

void childHandler(int sig) {
    if(sig == SIGUSR1) {
        int t = time(NULL);
        char symbol = (t%2==0 ? 'X' : 'O');

        int r = rand()%9;
        while(board[r] != ' ') r = (r+1)%9;
        board[r] = symbol;

        printf("Child plays %c at %d\n", symbol, r);
        printBoard();

        kill(getppid(), (symbol=='X' ? SIGUSR1 : SIGUSR2));
    }

    // GAME OVER SIGNAL CASE
    if(sig == SIGUSR2) {
        int t = time(NULL);

        // even → replay with X | odd → replay with O
        if(t % 2 == 0) {
            printf("Child requests replay, child wants X first.\n");
            kill(getppid(), SIGUSR1);
        } else {
            printf("Child requests replay, child wants O first.\n");
            kill(getppid(), SIGUSR2);
        }
    }
}

```

```

    }
}

void parentReplayHandler(int sig) {
    parent_signal = sig;
}

int main() {
    srand(time(NULL));
    child_pid = fork();

    if(child_pid == 0) {
        signal(SIGUSR1, childHandler);
        signal(SIGUSR2, childHandler);

        while(1) pause();
    }

    // Parent setup
    signal(SIGUSR1, parentReplayHandler);
    signal(SIGUSR2, parentReplayHandler);

    while(1) {
        parent_signal = 0;
        resetBoard();
        printf("\n--- New Game Started ---\n");

        while(1) {
            parentMove();
            if(hasWinner() || boardFull()) break;

            kill(child_pid, SIGUSR1);

            pause(); // wait child's move

            if(hasWinner() || boardFull()) break;
        }

        printf("Game finished.\n");

        // tell child game is over
        kill(child_pid, SIGUSR2);
    }
}

```

```

    // wait for replay request
    pause();

    if(parent_signal == SIGUSR1)
        printf("Parent: Child wants replay → child plays X\n");
    else if(parent_signal == SIGUSR2)
        printf("Parent: Child wants replay → child plays O\n");
    }

    return 0;
}

```

Output :➡

```

--- New Game Started ---

Parent plays X at 3
X
X

Child plays O at 5
X
X O

Parent plays X at 4
X
XXO

Child plays O at 2
X O
XXO

Parent plays X at 6
X O
XXO
X

Parent wins!
Game finished.
Child requests replay, child wants X first.
Parent: Child wants replay → child plays X

```